Dossier _ Tecnologías

El análisis arquitectónico de interfaces móviles exige comprender cómo cada capa tecnológica interactúa con el hilo principal (Main Thread) del dispositivo, el motor de renderizado y las APIs del sistema operativo, impactando directamente en la experiencia final. *(Contexto curricular: RA*

1. Arquitecturas Base: Desarrollo Nativo vs. Híbrido Web

La divergencia principal radica en el motor de renderizado, la gestión de la concurrencia y el acceso al hardware.

- **Rendimiento y Concurrencia:** Las **apps nativas** (Swift/Kotlin) compilan a código máquina (AOT - Ahead of Time) y gestionan la concurrencia mediante multithreading real a nivel de SO (Grand Central Dispatch en iOS, Coroutines en Android). Las **apps híbridas** (WebView) dependen del motor JavaScript (ej. V8), que es *single-threaded*. La concurrencia en híbridas se basa en el *Event Loop* (asincronía no bloqueante) y requiere delegar tareas pesadas a *Web Workers* para no bloquear el renderizado del UI Thread, evitando caídas de *frames*.
- **UX/UI y Renderizado:** El entorno nativo dibuja directamente utilizando las primitivas gráficas del sistema (Metal/Vulkan) a 60-120 FPS estables. El enfoque híbrido clásico (WebView) manipula el DOM, provocando costosos procesos de *Reflow* y *Repaint* en el motor del navegador (WebKit/Blink) ante animaciones complejas, lo que puede generar *stuttering* (tirones visuales).
- **Accesibilidad (a11y):** En nativo, los elementos de UI se exponen directamente al árbol de accesibilidad del SO (VoiceOver/TalkBack). En arquitecturas híbridas, el DOM actúa como intermediario, requiriendo un uso estricto y manual de semántica HTML y etiquetas WAI-ARIA para que el lector de pantalla interprete correctamente los nodos.
- **Seguridad:** El nativo cuenta con protección a nivel de binario y acceso directo a *Secure Enclave/Keystore*. Las híbridas se ejecutan en un entorno *Sandbox* (WebView) y su principal vector de ataque es el XSS (Cross-Site Scripting), requiriendo sanitización estricta del DOM y configuraciones seguras en los plugins de comunicación (*Bridge*) como Capacitor o Cordova.

2. Paradigmas Multiplataforma: Flutter vs. Ionic

Cuando evitamos el desarrollo nativo puro, el mercado se divide en dos aproximaciones radicales para la generación de la UI *(Contexto curricular: RA 1, RA 3)*.

Aspecto Arquitectónico	**Ionic (Capacitor / WebView)**	**Flutter (Dart / Skia o Impeller)**
Motor de Renderizado (UI/UX)	Delega el dibujado al motor web del SO. Emula la UI nativa (Adaptive Styling) inyectando CSS específico de plataforma.	Ignora el SO. Dibuja cada píxel de forma independiente utilizando su propio motor gráfico nativo (compilado en C++).
Rendimiento Base	Limitado por el DOM y el puente de comunicación JS-Nativo (aunque Capacitor serializa de forma eficiente).	Altísimo. Compila AOT a binario ARM nativo. Garantiza 60/120 FPS al no depender de WebViews ni *Bridges* en tiempo real.
Concurrencia	JS Event Loop + Promesas/Async-Await.	Utiliza *Isolates* (hilos de memoria independientes sin estado compartido), logrando concurrencia paralela real sin bloqueos.
Accesibilidad	Basada en los estándares web (WCAG, ARIA).	Renderiza un árbol de semántica (Semantics Tree) paralelo que mapea sus *widgets* a las APIs de accesibilidad del SO.

3. Motores de Reactividad SPA: Angular vs. React vs. Vue

Dentro del ecosistema Ionic (o cualquier entorno basado en WebView/DOM), la elección del framework JavaScript define cómo se interceptan los cambios de estado y cómo se actualiza la interfaz *(Contexto curricular: RA 2 - Generación declarativa; RA 3 - Eventos)*.

React (Ecosistema Funcional y Concurrente)

- **Reactividad:** Utiliza un enfoque funcional basado en *Virtual DOM* y un flujo de datos unidireccional. No observa mutaciones en las variables, sino que exige el reemplazo de estados mediante *Hooks* (ej. useState). Al detectar un nuevo estado, calcula la diferencia (algoritmo de *Reconciliation*) y parchea solo los nodos DOM modificados.
- **Asincronía y Concurrencia:** A partir de React 18, integra **Renderizado Concurrente**. Permite interrumpir la creación de la UI en memoria para atender eventos urgentes del usuario (como un input de teclado), evitando el bloqueo del hilo principal. Utiliza Suspense para la orquestación declarativa de datos asíncronos.
- **Seguridad:** El parser de su sintaxis declarativa (JSX) inyecta y escapa por defecto todas las variables interpoladas como strings, neutralizando de raíz ataques de inyección XSS.

Angular (Ecosistema Reactivo Completo)

- **Reactividad:** Históricamente dependiente de Zone.js, que intercepta parcheando eventos globales asíncronos (clics, temporizadores, peticiones HTTP) para desencadenar el *Change Detection* en el árbol de componentes. (Actualmente transicionando hacia un sistema de dependencias granulares con *Signals* para mejorar el rendimiento).
- **Asincronía:** Es su punto más fuerte a nivel de arquitectura empresarial gracias a la integración nativa de **RxJS**. Los observables permiten manejar la concurrencia asíncrona avanzada: cancelación de peticiones en vuelo, *debouncing*, *throttling* y flujos de datos continuos (WebSockets) de manera puramente funcional, algo que en React requiere bibliotecas de terceros o configuraciones complejas con useEffect.
- **Curva y Estructura:** Impone TypeScript estricto, Inyección de Dependencias (DI) y patrones decoradores, generando una arquitectura muy acoplada a las directrices de Google, pero extremadamente sólida para equipos grandes.

Vue (Equilibrio Reactivo)

- **Reactividad:** Utiliza el sistema de *Proxies* de ES6. Cuando un estado (ref/reactive) se renderiza, Vue registra el componente como dependencia. Al mutar el estado, el Proxy intercepta la escritura y dispara el re-renderizado exclusivamente del componente afectado, sin necesidad de calcular un árbol completo como React ni usar Zone.js como Angular.
- **Rendimiento y UX:** Al tener un Virtual DOM altamente optimizado y un sistema de reactividad reactivo por naturaleza (*fine-grained*), es extremadamente eficiente en memoria y velocidad inicial en entornos WebView, superando frecuentemente a React en rendimiento puro (sin necesidad de *memoization* manual como useMemo o useCallback).